

FIAP — Faculdade de Informática e Administração Paulista

DevOps e Arquitetura Cloud

ToggleMaster: de Monólito a um Ecossistema de Microsserviços Orquestrado em Kubernetes (AWS EKS)

Relatório de Entrega — Tech Challenge Fase 2

Karlos Rodrigues — RM [\[preencher\]](#) — Discord: [\[preencher\]](#)

Sumário

1. Introdução e Contexto do Projeto	3
2. Arquitetura da Solução	3
3. Containerização com Docker	5
4. Infraestrutura Provisionada na AWS	8
5. Orquestração com Kubernetes	10
6. Ingress e Acesso Externo	11
7. Escalabilidade Automática (HPA)	12
8. Desafios e Decisões Técnicas	13
9. Considerações Finais e Links do Projeto	16
Apêndice A — Evidências de Execução	17
Apêndice B — De-para: Requisitos do Enunciado × Entregas	17

1. Introdução e Contexto do Projeto

Na Fase 1, o ToggleMaster foi entregue como um MVP monolítico: uma aplicação Flask com PostgreSQL, publicada numa EC2 com banco RDS privado. Já naquela entrega, a análise do monolito apontava candidatos naturais a serem extraídos como serviços independentes — um de avaliação de flags, de leitura pesada e que se beneficiaria de cache próprio; um de administração das flags; e um de autenticação. É essa separação que a Fase 2 executa.

O motivo é o gargalo que já era esperado desde a Fase 1: escalabilidade grosseira, ou seja, não dá para escalar só o caminho de leitura sem escalar a aplicação inteira junto. Com a plataforma validada e a base de clientes crescendo, a diretoria da DevOps Solutions Inc. decidiu reescrever o sistema como um conjunto de microsserviços distribuídos, orquestrados em Kubernetes.

O trabalho desta fase foi containerizar os cinco microsserviços entregues pela equipe de arquitetura (`auth-service`, `flag-service`, `targeting-service`, `evaluation-service` e `analytics-service`), provisionar a infraestrutura de nuvem que eles precisam, e implantar tudo num cluster Kubernetes com um único ponto de entrada e escala automática.

Este projeto seguiu a Opção B do enunciado — conta pessoal AWS. O cluster foi criado com `eksctl create cluster`, que já cuida de gerar as IAM roles corretas sozinho, sem depender de uma role fixa como a `LabRole` do AWS Academy. Isso deixou o caminho livre para usar IRSA sem restrição, tanto no Ingress Controller quanto, de forma bem mais granular, nos service accounts do `evaluation-service` e do `analytics-service` (Seção 5.3).

1.1 Ambiente de desenvolvimento

Para não misturar o trabalho com a estação de trabalho pessoal, todo o desenvolvimento aconteceu numa VM dedicada (Ubuntu 24.04 sobre KVM/libvirt, 4 vCPU, 6 GB de RAM), provisionada inteira por cloud-init. Docker, kubectl, Helm, eksctl, AWS CLI e as ferramentas de carga já vêm junto — o ambiente inteiro é reconstruído do zero a partir de um único arquivo, sem passo manual nenhum.

2. Arquitetura da Solução

A solução tem duas camadas. O cluster Kubernetes orquestra os cinco microsserviços e expõe um ponto único de entrada pelo Ingress. Os serviços gerenciados da AWS cuidam da persistência, do cache e da mensageria. Nenhum estado vive dentro do cluster — é isso que mantém os pods descartáveis e permite escalá-los sem cuidado especial.

Microserviço	Responsabilidade	Persistência / Integração
auth-service (Go)	Autenticação e gestão de chaves de API	Amazon RDS (PostgreSQL) dedicado
flag-service (Python)	CRUD das definições de feature flags	Amazon RDS (PostgreSQL) dedicado
targeting-service (Python)	Regras de segmentação de público (JSONB)	Amazon RDS (PostgreSQL) dedicado
evaluation-service (Go)	Hot path — retorna a decisão true / false	Amazon ElastiCache (Redis), publica eventos no SQS
analytics-service (Python)	Consome eventos da fila e grava dados de análise	Amazon SQS (consumidor), Amazon DynamoDB

Tabela 1 — Responsabilidades e dependências de dados de cada microserviço.

2.1 Fluxo de uma requisição

O tráfego externo entra por um Load Balancer que o Nginx Ingress Controller provisiona sozinho, e é roteado por prefixo até o Service `clusterIP` correspondente. O caminho mais importante é o do `evaluation-service`: primeiro consulta o Redis, e se der cache miss, busca ao mesmo tempo a definição da flag no `flag-service` e a regra de segmentação no `targeting-service`, junta os dois resultados, grava no cache com TTL de 30 segundos e responde. A decisão só depois é publicada, de forma assíncrona, na fila SQS — sem segurar a resposta ao cliente por causa disso.

A avaliação em si é determinística: o serviço calcula um bucket de 0 a 99 a partir do hash SHA-1 de `user_id + flag_name`. O mesmo usuário sempre recebe a mesma resposta para a mesma flag, sem guardar estado por usuário em lugar nenhum — o que é essencial para que qualquer réplica possa atender qualquer requisição, indiferente de qual pod respondeu da última vez.

Cada serviço ficou isolado no seu próprio namespace, o que evita colisão de nomes de recursos e deixa RBAC e cotas configuráveis por área, de forma independente.

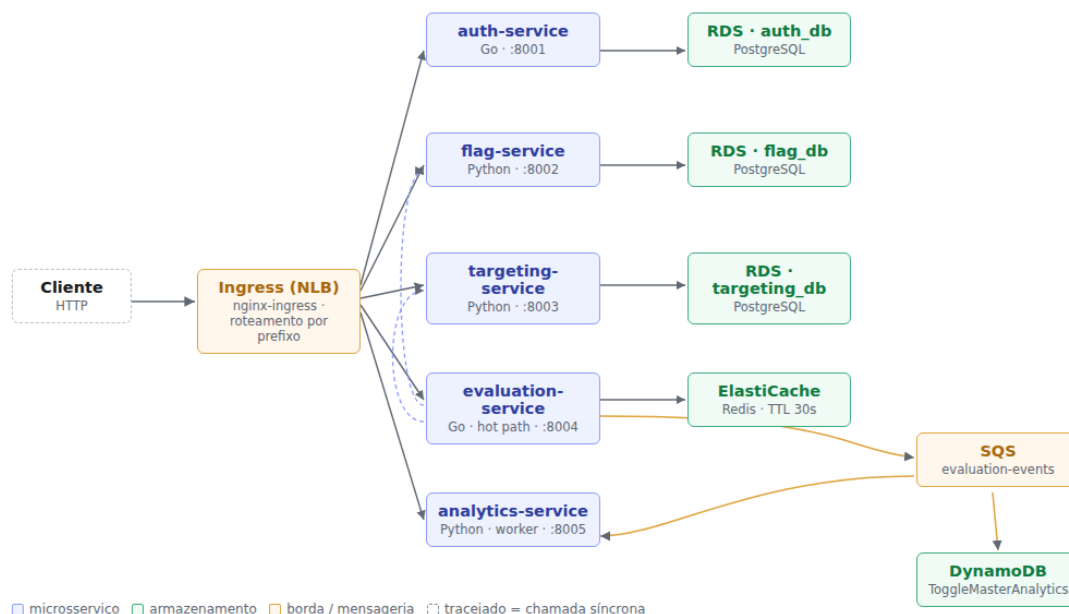


Figura 1 — Arquitetura do ToggleMaster na nuvem: os 5 microserviços, os 6 recursos de dados da AWS e o caminho do tráfego externo.

3. Containerização com Docker

Cada microserviço ganhou um Dockerfile próprio com build multi-stage, separando o estágio de compilação do estágio de execução — assim nenhuma ferramenta de build sobra na imagem final.

3.1 Estratégia por linguagem

Os serviços em Go compilam em `golang:1.21-alpine` e rodam a partir de `gcr.io/distroless/static-debian12:nonroot`. O binário é estático (`CGO_ENABLED=0`), com `-trimpath` tirando os caminhos da máquina de build e `-ldflags="-s -w"` descartando símbolos e informação de depuração. A imagem final não tem shell nem gerenciador de pacotes — não sobra nenhum binário além da própria aplicação, o que reduz bastante a superfície de ataque.

Os serviços em Python usam `python:3.11-slim` nos dois estágios: o primeiro monta um `virtualenv`, e só ele atravessa para o estágio final — o `pip` e os artefatos de compilação ficam para trás. A execução roda com um usuário não-root (`uid 10001`), sob Gunicorn.

```
# evaluation-service — Dockerfile multi-stage

FROM golang:1.21-alpine AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download          # camada separada: só invalida se as dependências mudarem
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build \
    -trimpath -ldflags="-s -w" \
    -o /out/evaluation-service .

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/evaluation-service /evaluation-service
EXPOSE 8004
USER nonroot:nonroot
ENTRYPOINT ["/evaluation-service"]
```

3.2 Imagens produzidas

Imagem	Tamanho final	Base do estágio de execução
auth-service	16,5 MB	distroless/static
evaluation-service	21,8 MB	distroless/static
flag-service	248 MB	python:3.11-slim
targeting-service	248 MB	python:3.11-slim
analytics-service	287 MB	python:3.11-slim

Tabela 2 — Tamanho das imagens após o build multi-stage.

A diferença de mais de dez vezes entre Go e Python é esperada: o binário Go é autocontido e não precisa de runtime, enquanto as imagens Python carregam o interpretador e a biblioteca padrão junto.

3.3 Ambiente local — Docker Compose

O `docker-compose.yml` na raiz do repositório sobe os nove contêineres que o enunciado pede: os cinco microsserviços e os quatro bancos.

Contêiner	Porta	Função
auth-service	8001	Chaves de API
flag-service	8002	CRUD das feature flags
targeting-service	8003	Regras de segmentação
evaluation-service	8004	Hot path da decisão
analytics-service	8005	Worker SQS → DynamoDB
postgres-auth	—	Banco auth_db
postgres-flags	—	Bancos flag_db e targeting_db
redis	—	Cache do evaluation-service
dynamodb-local	—	Tabela de eventos de analytics

Tabela 3 — Os nove contêineres do ambiente local.

O enunciado pede duas instâncias PostgreSQL locais para três serviços, então `postgres-flags` hospeda dois bancos logicos, criados já na inicialização do volume. Os schemas são montados direto dos próprios repositórios (`services/*/db/init.sql`) em vez de copiados para dentro do compose, o que evita ter duas versões do mesmo schema que podem sair de sincronia. Na AWS a separação passa ser física, como a Seção 4 exige.

Três decisões de configuração merecem registro:

- Os serviços de aplicação só sobem depois que os bancos reportam `healthy` (`depends_on` com `condition: service_healthy`), o que evita falha de corrida no boot.
- O `analytics-service` roda com um único worker de propósito. O consumidor SQS inicia no import do módulo, então cada worker do Gunicorn abriria um consumidor independente da mesma fila dentro do mesmo pod — com `--workers 1` fica valendo a relação 1 pod = 1 consumidor, e a contagem de réplicas passa a ser proporcional à vazão real de mensagens. É essa premissa que sustenta a escala automática da Seção 7.
- A tabela do DynamoDB Local é criada por um contêiner efêmero, sob um profile dedicado, que executa e encerra — não entra na contagem dos nove contêineres.

3.4 Validação do ambiente local

Depois de `docker compose up -d --build`, os nove contêineres sobem e os bancos reportam `healthy`. O fluxo foi testado de ponta a ponta:

Verificação	Resultado obtido
Health dos serviços	<code>{"status": "ok"}</code> nas portas 8001, 8002, 8003 e 8004
Criação de chave de API	<code>POST /admin/keys</code> autenticado com a <code>MASTER_KEY</code> retornou uma chave <code>tm_key_...</code> de 71 caracteres
Criação da feature flag	<code>enable-new-dashboard</code> criada com <code>is_enabled: true</code>
Criação da regra	<code>{"type": "PERCENTAGE", "value": 50}</code> gravada como JSONB
Avaliação de 8 usuários	4 resultados <code>true</code> e 4 <code>false</code> — bate com a regra de 50%, e o resultado é determinístico por <code>user_id + flag_name</code>
Cache	Chave <code>flag_info:enable-new-dashboard</code> presente no Redis; o log confirma <code>Cache MISS</code> seguido de <code>Cache HIT</code>
DynamoDB Local	Tabela criada com chave de partição <code>event_id</code> (String)

Tabela 4 — Validação funcional do ambiente local.

```

udefault@forge: ~/fiap-tc-fase2 - docker compose ps

SERVICE                STATUS
analytics-service       Up 57 minutes (healthy)
auth-service             Up 3 days
dynamodb-local           Up 3 days
evaluation-service       Up 3 days
flag-service             Up 3 days (healthy)
postgres-auth            Up 3 days (healthy)
postgres-flags           Up 3 days (healthy)
redis                   Up 3 days (healthy)
targeting-service        Up 3 days (healthy)

```

Figura 2 — Ambiente local completo em execução, captura real de `docker compose ps`.

4. Infraestrutura Provisionada na AWS

Toda a infraestrutura foi provisionada via AWS CLI (Opção B), na região `us-east-1`. Os nós do cluster usam a IAM Role `eksctl-togglemaster-nodegroup-togg-NodeInstanceRole`, e há mais duas roles dedicadas via IRSA (Seção 5.3) para os pods que precisam falar com SQS e DynamoDB.

Recurso	Qtd.	Nome / Identificador	Configuração
Cluster Kubernetes	1	<code>togglmaster</code>	EKS 1.31 · us-east-1 · criado via <code>eksctl create cluster --with-oidc</code>
Managed Node Group	1	<code>togglmaster-nodes</code>	<code>t3.medium</code> · Min=1 / Desejado=2 / Máx=4
Repositórios ECR	5	um por microserviço, tag <code>v1</code>	Imagens da Seção 3 publicadas e confirmadas com <code>aws ecr list-images</code>
RDS PostgreSQL	3	<code>togglmaster-auth</code> · <code>togglmaster-flag</code> · <code>togglmaster-targeting</code>	<code>db.t3.micro</code> · PostgreSQL 16.4 · Single-AZ · <code>--no-publicly-accessible</code>
ElastiCache Redis	1	<code>togglmaster-evaluation-cache</code>	<code>cache.t3.micro</code> · Redis 7.1
Tabela DynamoDB	1	<code>ToggleMasterAnalytics</code>	Chave de partição <code>event_id</code> (String) · billing <code>PAY_PER_REQUEST</code>
Fila SQS Standard	1	<code>togglmaster-evaluation-events</code>	Produtor: <code>evaluation-service</code> · Consumidor: <code>analytics-service</code>

Tabela 5 — Inventário dos recursos de nuvem provisionados na Etapa 2 (todos `available` / `ACTIVE` confirmados via AWS CLI).

O `eksctl`, sem `--vpc-*` explícito, provisiona uma VPC nova para o cluster — neste caso `192.168.0.0/16` — diferente da VPC padrão da conta (`172.31.0.0/16`), que é onde o RDS e o ElastiCache tinham sido criados. Isso só apareceu na prática quando os pods começaram a dar `i/o timeout` tentando falar com o Redis: eram duas redes isoladas, sem rota entre si. Como os blocos CIDR não colidem, a saída foi VPC Peering entre as duas, com rota adicionada nas route tables dos dois lados (a rota inversa na VPC padrão, e a rota direta na route table que as duas subnets públicas dos nós do EKS compartilham — por padrão o `eksctl` não usa `--node-private-networking`). Numa próxima rodada, apontar o `eksctl` direto para as subnets da VPC padrão via `--vpc-private-subnets` / `--vpc-public-subnets` evitaria esse peering todo.

Com `--no-publicly-accessible` (correto, como o enunciado pede), os endpoints do RDS só resolvem DNS de dentro da VPC — o que é esperado, mas teve uma consequência prática: rodar as migrações de schema (`db/init.sql` de cada serviço) exigiu subir um pod temporário com `postgres:16-alpine` dentro do próprio cluster (`kubectl run` + `kubectl cp` + `kubectl exec -- psql`), já que não dava para conectar direto da máquina de desenvolvimento.

Depois que cada recurso foi criado, os endpoints, nomes de tabela e ARNs foram anotados e injetados no cluster como Secrets e ConfigMaps (Seção 5) — nenhuma credencial fica no código-fonte nem nas imagens publicadas no ECR. Vale registrar que o nome da tabela e a chave de partição do DynamoDB não são escolha livre: o `analytics-service` grava o item usando `event_id` como partition key e lê o nome da tabela da variável `AWS_DYNAMODB_TABLE`, então a tabela provisionada precisa respeitar esse contrato — senão o worker só falha em runtime, ao tentar gravar o primeiro evento.

5. Orquestração com Kubernetes

Cada microsserviço ficou isolado no seu próprio namespace, com os manifestos de Deployment, Service (ClusterIP), Secret e ConfigMap aplicados via `kubectl apply`. As imagens referenciadas nos Deployments apontam para os repositórios do ECR publicados na Seção 4, tag `v1`. No estado final, os 9 pods ficaram Running e Ready nos 5 namespaces.

5.1 Estrutura das Secrets

Os valores reais não entram neste relatório nem no repositório versionado — só a estrutura do manifesto:

```
apiVersion: v1
kind: Secret
metadata:
  name: auth-db-credentials
  namespace: auth
type: Opaque
data:
  DATABASE_URL: [base64 – definido em tempo de deploy]
  MASTER_KEY:   [base64 – definido em tempo de deploy]
```

Todas as Secrets seguem a codificação em base64 que o Kubernetes exige, geradas a partir de um arquivo `.env` local que não é versionado (fica fora via `.gitignore`) e só vira YAML no momento do deploy.

5.2 Boas práticas aplicadas em todos os Deployments

- Requests e Limits de CPU e memória em todos os contêineres, para um pod não consumir recursos além do necessário e atrapalhar o agendamento dos outros no mesmo nó.
- Readiness e Liveness Probes via `GET /health`, presente nos cinco serviços. O `initialDelaySeconds` acompanha o tempo de boot de cada linguagem: os binários Go respondem quase na hora, enquanto os serviços Python sob Gunicorn precisam de uma folga maior.
- Separação por Namespace, o que permite aplicar `ResourceQuota` de forma independente por serviço.
- Um worker por pod no `analytics-service`, pelo motivo já explicado na Seção 3.3 — é o que torna a contagem de réplicas uma medida fiel da capacidade de consumo da fila.

5.3 Acesso a SQS e DynamoDB — IRSA com política de menor privilégio

O `evaluation-service` e o `analytics-service` precisam de credenciais AWS para falar com SQS e DynamoDB. A role padrão dos nós do EKS (`AmazonEKSWorkerNodePolicy` + `AmazonEC2ContainerRegistryPullOnly`) não traz essas permissões, e isso é proposital: dar essa permissão ao node inteiro faria qualquer pod agendado ali herdar o acesso, o que violaria o princípio do menor privilégio.

A saída foi IRSA (IAM Roles for Service Accounts), possível porque o cluster nasceu com `--with-oidc`. Duas service accounts dedicadas (`evaluation-sa`, `analytics-sa`) foram criadas via `eksctl create iamserviceaccount`, cada uma com sua própria IAM Role, anexadas a uma policy com três declarações separadas em vez de uma permissão genérica de SQS/DynamoDB:

```
{
  "Statement": [
    { "Sid": "SqsEvaluationProducer",
      "Action": ["sqs:SendMessage"], "Resource": "arn:aws:sqs:...:togglemaster-evaluation-
events" },
    { "Sid": "SqsAnalyticsConsumer",
      "Action": ["sqs:ReceiveMessage", "sqs:DeleteMessage", "sqs:GetQueueAttributes"],
      "Resource": "arn:aws:sqs:...:togglemaster-evaluation-events" },
    { "Sid": "DynamoDbAnalytics",
      "Action": ["dynamodb:PutItem"], "Resource": "arn:aws:dynamodb:...:table/
ToggleMasterAnalytics" }
  ]
}
```

O `evaluation-service` só consegue enviar mensagens, porque é o produtor. O `analytics-service` só consegue receber e apagar, porque é o consumidor, e só pode fazer `PutItem` na tabela específica — nunca `Scan`, `Query` ou qualquer leitura em massa. Cada Deployment referencia a sua service account em `spec.template.spec.serviceAccountName`.

6. Ingress e Acesso Externo

O Nginx Ingress Controller foi instalado via Helm (`ingress-nginx/ingress-nginx`), com o Service do controller do tipo `LoadBalancer` anotado para provisionar um Network Load Balancer (`service.beta.kubernetes.io/aws-load-balancer-type: nlb`). Quem provisiona o NLB é o cloud-controller-manager nativo do EKS, então não foi preciso instalar o AWS Load Balancer Controller à parte. O manifesto Ingress roteia por prefixo de rota:

Rota	Service de destino	Porta
/auth	auth-service	8001
/flags	flag-service	8002
/targeting	targeting-service	8003
/evaluate	evaluation-service	8004
/events	analytics-service	8005

Tabela 6 — Regras de roteamento do manifesto Ingress.

O hostname do NLB provisionado é `abc9bb709ba2943ea8099ae621c5939a-76d2e22c4f7f64c3.elb.us-east-1.amazonaws.com`. A validação de acesso externo chamou as quatro rotas por curl:

```
$ curl http://abc9bb709ba2943ea8099ae621c5939a-76d2e22c4f7f64c3.elb.us-east-1.amazonaws.com/
auth/health
{"status":"ok"}
$ curl http://abc9bb709ba2943ea8099ae621c5939a-76d2e22c4f7f64c3.elb.us-east-1.amazonaws.com/
flags/health
{"status":"ok"}
$ curl http://abc9bb709ba2943ea8099ae621c5939a-76d2e22c4f7f64c3.elb.us-east-1.amazonaws.com/
targeting/health
{"status":"ok"}
$ curl http://abc9bb709ba2943ea8099ae621c5939a-76d2e22c4f7f64c3.elb.us-east-1.amazonaws.com/
evaluate/health
{"status":"ok"}
```

As quatro rotas responderam HTTP 200 passando pelo balanceador de verdade, não só por port-forward, o que confirma que o rewrite-target descrito abaixo está funcionando: o Ingress recebe `/auth/health` e o `auth-service` recebe só `/health`, que é o que o código-fonte espera.

As rotas dos serviços são `/health`, `/flags/<nome>`, `/rules/<flag>` e `/evaluate`, sem o prefixo do Ingress. Sem reescrever o caminho (`rewrite-target`), o backend responde 404 para todo o tráfego que passa pelo balanceador — mesmo funcionando perfeitamente quando testado por port-forward, o que faz o problema passar despercebido até chegar no ambiente real.

7. Escalabilidade Automática (HPA)

O Metrics Server foi instalado como addon gerenciado do EKS (o achado da Seção 8.3 explica por quê) e confirmado com `kubectl top nodes` antes de qualquer teste de carga.

Serviço	Métrica	Alvo	Mín.	Máx.
<code>evaluation-service</code>	Utilização média de CPU	70%	2	6
<code>analytics-service</code>	Utilização média de CPU	65%	1	5

Tabela 7 — Configuração dos HorizontalPodAutoscalers, ambos aplicados e ativos no cluster.

7.1 Teste de carga — evaluation-service

A carga foi gerada com `ab` (ApacheBench) direto na rota `/evaluate` do NLB, simulando tráfego real de cliente externo, não port-forward:

```
$ ab -n 200000 -c 60 -t 240 "http://<NLB>/evaluate/evaluate?user_id=load-user&flag_name=enable-new-dashboard"
```

Momento	CPU (uso/alvo)	Réplicas
Antes da carga (baseline)	1% / 70%	2
Sob carga (pico observado)	273% / 70%	4
Após o fim da carga (cooldown de 180s)	1% / 70%	2 (scale-down automático)

Tabela 8 — Comportamento do HPA do `evaluation-service` sob carga real, medido com `kubectl top pods` e `kubectl get hpa`.

O HPA reagiu dentro da janela de `stabilizationWindowSeconds: 0` configurada para scale-up, dobrando as réplicas de 2 para 4 pouco depois do início da carga, e voltou ao piso de 2 depois do período de estabilização de scale-down, sem oscilar entre os dois estados.

7.2 Teste do fluxo assíncrono — analytics-service

Em vez de mandar mensagens manuais pelo Console, o teste seguiu o caminho real do sistema: requisições de avaliação passando pelo Ingress, cada uma disparando um evento assíncrono para o SQS.

Passo	Resultado
20 requisições a <code>/evaluate/evaluate</code> via NLB	200 OK em todas, cada uma publicando 1 evento no SQS de forma assíncrona (<code>go a.sendEvaluationEvent</code>)
Consumo pelo <code>analytics-service</code>	Long-polling de até 20s, sem intervenção manual
Itens gravados no DynamoDB	20 de 20, confirmado com <code>aws dynamodb scan --select COUNT</code>

Tabela 9 — Validação do fluxo `evaluation-service` → SQS → `analytics-service` → DynamoDB, de ponta a ponta na infraestrutura real.

Vale explicar por que a métrica de escala do `analytics-service` importa: o HPA por CPU mede o efeito (o processamento gasto ao tratar as mensagens), não a causa (a profundidade da fila). Funciona, mas reage só depois que a carga já virou consumo de CPU, e nunca escala a zero. O KEDA, observando `ApproximateNumberOfMessagesVisible` direto, escala de 0 a N e é a métrica mais correta para uma carga orientada a eventos. Essa distinção só se sustenta, aliás, por causa da decisão de `--workers 1` da Seção 3.3 — com vários consumidores por pod, nem a profundidade da fila nem a CPU seriam proporcionais ao número de réplicas.

8. Desafios e Decisões Técnicas

8.1 O código-fonte recebido não compilava

O primeiro passo do enunciado é rodar o ecossistema localmente. Ao clonar os cinco repositórios de `github.com/FIAP-TCs`, os dois serviços escritos em Go simplesmente não compilavam. Não é questão de estilo — são erros que o compilador rejeita, o que impede qualquer `go build` e, por tabela, o build das imagens. Os três serviços em Python foram usados sem alteração no código.

#	Arquivo	Defeito e correção
1	auth-service/go.mod	require github.com/jackc/pgx/v4/stdlib v4.18.3 — stdlib é um pacote dentro do módulo pgx/v4, não um módulo próprio. O Go recusa com version "v4.18.3" invalid: should be v0 or v1, e trava todo comando go. Correção: linha removida.
2	auth-service/main.go	Driver pgx/v4/stdlib importado sem _, mas nunca referenciado — só é usado pelo efeito colateral do init(). Correção: import em branco.
3	auth-service/main.go	"fmt" importado e não usado. Correção: removido.
4	auth-service/handlers.go	"crypto/sha256" e "encoding/hex" importados e não usados — quem os usa é key.go. Correção: removidos.
5	auth-service/key.go	"fmt" importado e não usado. Correção: removido.
6	evaluation-service/go.sum	O arquivo é uma cópia do go.mod (começa com module evaluation-service). O Go recusa com malformed go.sum: wrong number of fields 2. Correção: descartado e regenerado com go mod tidy.
7	evaluation-service/evaluator.go	Usa os.Getenv("SERVICE_API_KEY") sem importar "os" (undefined: os). Correção: import adicionado.
8	evaluation-service/evaluator.go	"context" importado e não usado — o ctx das chamadas ao Redis é a variável global de main.go. Correção: removido.

Tabela 10 — Defeitos de compilação corrigidos no código-fonte recebido.

Depois das correções, `go build ./...` conclui sem erro nos dois serviços. O diagnóstico completo de cada defeito está no arquivo `PATCHES.md`, na raiz do repositório.

8.2 Dependências declaradas sem travas suficientes

Dois problemas de dependência impediriam a subida dos contêineres, e os dois foram resolvidos sem tocar no código-fonte nem no `requirements.txt` original:

- Flask 2.2.2 sem pin de Werkzeug. O `requirements.txt` fixa `Flask==2.2.2`, mas não fixa o Werkzeug. Como o Flask 2.2.2 só declara `Werkzeug>=2.2.2`, o pip resolve para a série 3.x, que removeu `werkzeug.urls.url_quote` — import que o Flask 2.2 ainda faz. O contêiner morreria com `ImportError` antes do primeiro request. A solução foi um `constraints.txt` travando `Werkzeug<3`, que complementa o arquivo original em vez de substituí-lo.
- boto3 velho demais para o DynamoDB Local. O `app.py` do `analytics-service` cria os clientes com `session.client("dynamodb")`, sem `endpoint_url` e sem nenhuma forma de configurá-lo por fora. As variáveis `AWS_ENDPOINT_URL_DYNAMODB` e `AWS_ENDPOINT_URL_SQS` resolveriam, mas só passaram a ser lidas pelo botocore a partir da série 1.31, e o `requirements.txt` fixa a 1.26.50. A solução foi um `requirements-container.txt`, instalado depois do original, elevando o boto3 — a mesma imagem serve os dois ambientes: localmente a variável aponta para o DynamoDB Local, na AWS basta não definir ela.

8.3 Dependência circular no bootstrap

O `evaluation-service` precisa de uma chave de API para conversar com o `flag-service` e o `targeting-service`, mas essa chave só existe depois que o `auth-service` já estiver no ar — não tem como injetá-la antes da primeira subida. A solução foi documentar um bootstrap em duas etapas: subir a stack, gerar a chave, gravá-la e recriar só o `evaluation-service`.

Uma armadilha apareceu durante a validação: carregar o `.env` no shell (`set -a; . ./env`) antes de chamar o `docker compose` faz o serviço receber a chave vazia e levar 401 do `flag-service` — mesmo com o valor certo gravado no arquivo. A causa é a ordem de precedência do Docker Compose: variável já exportada no ambiente vence o arquivo `.env`. O sintoma engana, porque a mesma chave testada isolada com `curl` responde 200 sem problema. O diagnóstico veio de comparar o valor do arquivo com o que o contêiner de fato recebeu, via `docker inspect`.

8.4 HPA por CPU × KEDA por profundidade de fila

Numa conta pessoal, a escolha mais correta para o `analytics-service` seria o KEDA, escalando direto pela métrica `ApproximateNumberOfMessagesVisible` da fila SQS, evitando o atraso natural do HPA por CPU — que só reage depois que a carga já virou consumo de processamento. No ambiente AWS Academy o KEDA não é viável, porque depende de IRSA para criar roles próprias, e nesse caso vale o workaround que o próprio enunciado sugere: escalar por CPU. A discussão completa está na Seção 7.

8.5 Diferença de propósito entre os três data stores

- Amazon RDS (PostgreSQL) guarda os dados relacionais e transacionais dos serviços de domínio (`auth`, `flag`, `targeting`), onde consistência forte e relacionamento entre entidades são essenciais. É o caso do `targeting-service`, que guarda a regra como JSONB mas depende da unicidade de `flag_name` garantida pelo banco.
- Amazon ElastiCache (Redis) é o cache em memória de baixíssima latência para o hot path, com TTL de 30 segundos. Prioriza velocidade de leitura em troca de persistência — perder o cache custa só um miss, nunca um dado de verdade.
- Amazon DynamoDB guarda eventos de analytics em alto volume, com escrita desnormalizada, priorizando throughput e escala horizontal sobre consultas relacionais complexas. Cada evento é independente e nunca é atualizado, o que dispensa transação.

8.6 Isolamento do ambiente de desenvolvimento

Todo o trabalho foi feito numa VM dedicada, provisionada por cloud-init (Seção 1.1), em vez de instalar o ferramental direto na estação de trabalho. O ganho não é só organização: o ambiente é descartável e recriável, o que elimina a classe de problema em que algo funciona só por causa de um resíduo de configuração local que ninguém mais consegue reproduzir.

8.7 Achados reais do provisionamento (Opção B)

Achado	Como foi contornado
Versão de EKS do enunciado (1.29) fora de suporte	Erro <code>invalid version, 1.29 is no longer supported</code> . Usada a versão suportada mais próxima, 1.31 — o enunciado é de out/2025 e a AWS já descontinuou a série.
Conta no AWS Free Plan trava RDS em 2 instâncias	Não é a cota padrão de serviço (essa é 40), é uma restrição do <code>accountPlanType: FREE</code> . Erro: <code>InstanceQuotaExceeded... upgrade your account plan</code> . Resolvido com upgrade para <code>PAID</code> , sem custo imediato — os créditos gratuitos da conta continuaram intactos.
<code>eksctl</code> cria uma VPC isolada da VPC padrão	Pods sem rota para o RDS/ElastiCache, criados na VPC padrão. Resolvido com VPC Peering e rotas nos dois lados (detalhe na Seção 4).
Manifest comunitário do Metrics Server colide com o addon gerenciado do EKS	O EKS já instala o Metrics Server como addon nativo. Aplicar a manifest padrão do GitHub por cima trocou os labels do pod e quebrou o Service correspondente (<code>MissingEndpoints</code>). Corrigido removendo e recriando o addon gerenciado (<code>aws eks delete-addon / create-addon</code>) — a lição é nunca sobrepor um addon gerenciado com a manifest da comunidade.
RDS privado inacessível da máquina de desenvolvimento	Por desenho (<code>--no-publicly-accessible</code>). As migrações de schema rodaram de dentro do cluster, via um pod temporário com <code>psql</code> (Seção 4).
Node role sem permissão para SQS/DynamoDB	A role padrão dos nós do EKS não traz essas permissões, de propósito, para não dar acesso a todo pod agendado no nó. Resolvido com IRSA e política de menor privilégio (Seção 5.3), em vez de anexar a permissão na role do nó inteiro.

Tabela 11 — Achados reais durante o provisionamento na AWS, com a correção aplicada em cada um.

9. Considerações Finais e Links do Projeto

A migração para microsserviços concretizou o que o relatório da Fase 1 já apontava como candidato a extração — em especial separar o caminho de leitura de alta frequência, agora com cache próprio e escala independente, do CRUD administrativo das flags. Cada serviço passou a ter seu próprio ciclo de implantação, sua própria persistência e sua própria política de escala, o que resolve a escalabilidade grosseira que era a principal desvantagem do monolito.

O ganho mais fácil de enxergar está na containerização: imagens de 16,5 MB e 21,8 MB para os serviços em Go, sem shell nem gerenciador de pacotes, dão uma superfície de ataque e um tempo de pull muito menores do que uma imagem base convencional entregaria.

Mas o aprendizado que mais pesou não veio da orquestração — veio da base recebida. Oito defeitos de compilação e dois problemas de dependência precisaram ser resolvidos antes que qualquer contêiner subisse. Não adianta discutir estratégia de autoescala se o `go build` nem passa.

Item	Link
Repositório (código-fonte, compose e manifestos)	[preencher]
Forks dos 5 microsserviços com as correções da Seção 8.1	[preencher]
Vídeo de demonstração (até 20 min)	[preencher — YouTube]
Diagrama de arquitetura	Figura 1, Seção 2
Badge Google Cloud Skills Boost (pontuação extra)	[preencher — perfil público, se concluída]
Repositório da Fase 1 (referência)	https://github.com/dougls/toggle-master-monolith

Tabela 12 — Links da entrega.

Apêndice A — Evidências de Execução

Capturas comprovando cada etapa, a serem referenciadas pelos tempos correspondentes no vídeo.

Item	Evidência	Status
A.1	go build concluindo nos dois serviços Go após as correções	Disponível
A.2	docker images — tamanho das cinco imagens	Disponível
A.3	docker compose ps — os nove contêineres (Figura 2)	Disponível
A.4	Fluxo ponta a ponta: chave de API, flag, regra e avaliações	Disponível
A.5	Recursos provisionados na AWS — aws rds/elasticache/dynamodb/sqs describe-*	Disponível
A.6	Cluster ativo e kubectl get pods -A — 9/9 Running nos 5 namespaces	Disponível
A.7	Ingress: curl nas 4 rotas via hostname do NLB, todas 200	Disponível
A.8	Teste de carga (ab) e kubectl get hpa escalando de 2 para 4 réplicas	Disponível
A.9	20 avaliações via Ingress → SQS → analytics-service → 20 itens no DynamoDB	Disponível

Apêndice B — De-para: Requisitos do Enunciado × Entregas

Legenda: ✓ atendido · ~ parcial · ⚪ pendente

Requisito	Status	Onde
1 — Dockerfile otimizado (multi-stage) para os 5 microsserviços	✓	Seção 3.1, Tabela 2
1 — <code>docker-compose.yml</code> único com 5 apps + 4 bancos	✓	Seção 3.3, Tabela 3
1 — Ecossistema executando e compreendido localmente	✓	Seção 3.4, Tabela 4
2 — Cluster Kubernetes provisionado	✓	Seção 4
2 — 5 repositórios ECR com as imagens publicadas	✓	Seção 4
2 — 3 instâncias RDS PostgreSQL independentes	✓	Seção 4
2 — 1 cluster ElastiCache Redis	✓	Seção 4
2 — 1 tabela DynamoDB e 1 fila SQS Standard	✓	Seção 4
3 — Metrics Server instalado	✓	Seção 7
3 — Nginx Ingress Controller instalado	✓	Seção 6
4 — Manifestos: Namespace, Deployment, Service, Secret, ConfigMap	✓	Seção 5
4 — Ingress com roteamento por prefixo de rota	✓	Seção 6
4 — Requests/Limits, Secrets em base64, Probes e Namespaces	✓	Seção 5.2
5 — HPA no evaluation-service e no analytics-service	✓	Seção 7
Vídeo — compose com 9 contêineres, cluster, pods, Ingress e escala	🌀	Comandos prontos (Apêndice A); gravação pendente
Vídeo — explicar a escolha da métrica de escala do analytics-service	✓	Seções 7 e 8.4
Vídeo — explicar a diferença de propósito entre os 3 data stores	✓	Seção 8.5
Vídeo — explicar a arquitetura e os desafios encontrados	✓	Seções 2 e 8
Relatório — nome, RM e Discord dos participantes	~	Capa (RM e Discord a preencher)
Relatório — links dos repositórios e do vídeo	🌀	Seção 9, Tabela 9
Extra — trilha Google Cloud Skills Boost (10 pontos, opcional)	🌀	Seção 9, Tabela 9